El objeto `Array` nos permite **crear matrices de elementos**. A diferencia de JAVA, **el tamaño de los arrays** en JavaScript **puede variar** y, como en JAVA, **se accede a sus elementos a través de un índice** que empieza a contar desde 0.

Para crear un array, tenemos **tres opciones**:

- Emplear el operador `new Array([tam])` se creará un array con el tamaño inicial indicado
- Emplear el operador `new Array(lista)` en la que se creará el array con los elementos especificados
- Emplear un `Array` literal (conjunto de valores entre corchetes)

```
<script type="text/javascript">

    let a = new Array();

    a[0] = "uno";

    a[1] = "dos";

    a[2] = 124;

    let b = new Array("1","2","3");

    let c = ["c1","c2","c3"];

    for(i=0;i<a.length; i++){

        document.write(a[i] + " " + b[i] + " " + c[i] + "<br/>")

    }

</script>
```

La salida

```
uno 1 c1
```

```
dos 2 c2

124 3 c3
```

Podemos **añadir** elementos **al final** del array:

```
b[4] = "Otro";
```

Y pueden **mezclarse** elementos de diferentes tipos

```
b[5] = {uno:1,dos:2};
```

NOTA: Al igual que en JAVA **no disponemos de arrays multidimensionales,** pero lo que sí podemos **crear** es **arrays de arrays**

```
<script type="text/javascript">

    let a = new Array([1,2,3],[4,5]);

    for(let i=0;i<a.length;i++){

        for(let j=0;j<a[i].length;j++){

            document.write(a[i][j] + " ");

        }

        document.write("<br/>");

    }

</script>
```

La salida

```
1 2 3

4 5
```

Propiedades

- `lenght` : Retorna el **número de elementos** actuales de la matriz

Métodos

`concat(listaArrays)` : **Une** todos los **arrays** de la lista y los retorna como un nuevo array

`includes(valor)` : Retorna `true` si el `valor` está **en la matriz**

`indexOf(valor[,comienzo])` : Retorna la **posición** en la que se encuentra el elemento indicado a partir de la posición indicada en `comienzo` (o en 0 si no se especifica)

`fill(valor[,comienzo[,fin])` : **Rellena** los elementos de la matriz entre `comienzo` (o en 0 si no se especifica) y `fin` con el valor indicado

`find(funcion[,this])` : Se **evalúa la** `función` **para cada elemento** y, si retorna `true`, **se devuelve el elemento** (en caso contrario, retorna `undefined`). La función tiene como **parámetros** el **elemento sobre el que se está iterando**, el **índice** de dicho elemento y **la matriz** propiamente dicha **(opcionalmente** se puede pasar un argumento que representa qué va a ser `this` en la función callback)

`findIndex(funcion)` : Similar a la anterior pero **retorna la posición** (-1 si no la encuentra)

`filter(funcion)` : Retorna todos los **elementos** para los que la `función` retorna `true`.

`forEach(función)` : **Ejecuta** la `función` indicada por **cada miembro** de la matriz

`join()` : Retorna **todos los elementos del array como una cadena** (los elementos se separan por **comas)**

`map(funcion)` : Crea una **nueva matriz tras aplicar la función** indicada a cada **elemento**

`pop()` : **Elimina** el **último** elemento del array y lo retorna

`push(valor)` : **Añade** un **nuevo** elemento **al final** del array y retorna el nuevo tamaño (pueden añadirse varios a la vez)

`reverse()` : **Invierte** la posición de los elementos del array y lo retorna

`slice(inicio[,fin])` : Retorna un **nuevo array con los elementos** del array entre el `inicio` y el `fin` (o el final del array si no se indica `fin`)

`sort([función])` : **Ordena** los elementos del array. **Opcionalmente** se puede indicar la `función` a emplear para ordenar (dicha función debe **recibir dos parámetros** y retornar **un valor entero**). **NOTA:** Por **defecto** se ordena **alfabéticamente.**

`splice(inicial,numero[,elementos])` : Este método **añade o elimina** elementos del array a partir de la posición `inicial`. Se debe **indicar** el `número` de elementos **a eliminar**. **Opcionalmente** se pueden añadir **nuevos elementos en la posición indicada del arra**y. Si el valor `inicial` es **negativo** se empieza a **contar desde el final** de la matriz

`toString()` : Lo mismo que `join`

NOTA: método útil para convertir una cadena en un array, usando un separador

`split(delim)` : A partir de una cadena retorna un array con los elementos empleando el delimitador como separador entre ellos,

NOTA: Como ya hemos comentado **un array es un objeto** de Javascript. Lo que **lo diferencia de un objeto normal** es que los motores de Javascript **intentarán mantener los objetos en posiciones contiguas** de memoria al objeto de acelerar el acceso a los mismos. Estas **ventajas se pierden si añadimos propiedade**s a los arrays **o si dejamos huecos** en los mismos

For of

Además de poder recorrer una matriz con una repetitiva tradicional podemos emplear un `for of`

```
let arr = [1,2,3,4,5,6];

for(let v of arr){

    console.log(v);

}

>1

>2

>3

>4

>5

>6
```

Vemos ejemplos de uso de los métodos vistos:

```
let arr=[{id:1, nombre:'Juan',edad:18},

{id:2, nombre:'Ana',edad:21},

{id:3, nombre:'Eva',edad:19},

{id:4, nombre:'Carlos',edad:18}];
```

Añadir y extraer elementos

```
arr.push({id:5,nombre:'Luis',edad:23});

> 5
```

```
arr.pop(); //Borra el último elemento del array

> {id: 5, nombre: "Luis", edad: 23}
```

```
>(6) [{id: 1, nombre: "Juan", edad: 18}, {id: 2, nombre: "Ana", edad: 21}, {id: 3, nombre: "Eva", edad: 19}, {id: 4, nombre: "Carlos", edad: 18}, {id: 0, nombre: "Marta", edad: 21}, {id: 5, nombre: "Luis", edad: 23}]

arr.splice(2,0,{id:5,nombre:'Julio',edad:19});  //Inserción de elementos desde la posición inicial indicada

arr

> (5) [{id: 1, nombre: "Juan", edad: 18}, {id: 2, nombre: "Ana", edad: 21},

{id: 5, nombre: "Julio", edad: 19}, {id: 3, nombre: "Eva", edad: 19}, {id: 4, nombre: "Carlos", edad: 18}]

arr.splice(2,1); //Eliminación de elementos en el array, indica la posición inicial a eliminar y el número de elementos a eliminar

> {id: 5, nombre: "Julio", edad: 19}

arr

(4) [{id: 1, nombre: "Juan", edad: 18},

{id: 2, nombre: "Ana", edad: 21}, {id: 3, nombre: "Eva", edad: 19},

{id: 4, nombre: "Carlos", edad: 18}]

let copia = arr.concat([{id: 0, nombre: "Marta", edad: 21},{id: 5, nombre: "Luis", edad: 23}]); // Concatena los elementos indicados al array

copia

>(6) [{id: 1, nombre: "Juan", edad: 18}, {id: 2, nombre: "Ana", edad: 21}, {id: 3, nombre: "Eva", edad: 19}, {id: 4, nombre: "Carlos", edad: 18}, {id: 0, nombre: "Marta", edad: 21}, {id: 5, nombre: "Luis", edad: 23}]
```

Buscar elementos

```
[1,2,2,3,4].includes(2)

>true


[1,2,2,3,4].includes(-2)

>false


[1,2,2,3,4].indexOf(2)

>1


[1,2,2,3,4].indexOf(2,2)

//(Nos busca el numero 2 desde la posición 2)

>2
```

>**arr.find , se evalua la** `función` **para cada elemento** y, si retorna `true`, **se devuelve el elemento** (en caso contrario, retorna `undefined`).

```
let usuario = arr.find((item,index,arr) => {
    return item.nombre=='Ana'
})
usuario
>{id: 2, nombre: "Ana", edad: 21}

usuario = arr.find((item,index,arr) => {
    return item.nombre=='ana'
})



usuario
>undefined

usuario = arr.findIndex((item,index,arr) => {
    return item.nombre=='Ana'
})

>1
```

```
usuario = arr.findIndex((item,index,arr) => {

    return item.nombre=='ana'

})



>-1

//arr.filter:  Retorna todos los elementos para los que la función retorna true.

let mas18 = arr.filter((item) => item.edad > 18)

>mas18

> (2) [{id: 2, nombre: "Ana", edad: 21}, {id: 3, nombre: "Eva", edad: 19}]
```

Recorrer una matriz

```
arr.forEach(function(item,index,array) {
    console.log(`${item.id} : ${index} : ${item.nombre}`);

});




> 1 : 0 : Juan
2 : 1 : Ana
3 : 2 : Eva
4 : 3 : Carlos
```

Transformación de matrices

```
//La función map , se utiliza para transformar una entrada en una salida. Siendo esta entrada cada iteracción del array
// devuelve un array de la misma longitud pero con los elementos transformados a lo que hayamos indicado en la expresión

let mayus = arr.map((item) => item.nombre.toUpperCase());

mayus

>(4) ["JUAN", "ANA", "EVA", "CARLOS"]



[1,4,11,23,2].sort()

//Lo ordena alfanumericamente

>(5) [1, 11, 2, 23, 4]



[1,4,11,23,2].sort((a,b) => a-b) // Ascendente: >0 si a>b, ==0 si a==b y <0 si a<b

>(5) [1, 2, 4, 11, 23]

[1,4,11,23,2].sort((a,b) => b-a)

>(5) [23, 11, 4, 2, 1]

//Con el slice copia el array original para que el sort no modifique el array original, sino la copia

let ord = arr.slice().sort((a,b) => a.nombre > b.nombre?1: a.nombre == b.nombre?0:-1)

ord

>[{id: 2, nombre: "Ana", edad: 21}, {id: 4, nombre: "Carlos", edad: 18}, {id: 3, nombre: "Eva", edad: 19}, {id: 1, nombre: "Juan", edad: 18}]
```

```
ord.reverse()

>[{id: 1, nombre: "Juan", edad: 18}, {id: 3, nombre: "Eva", edad: 19}, {id: 4, nombre: "Carlos", edad: 18}, , {id: 2, nombre: "Ana", edad: 21}]

//split(delim) : A partir de una cadena retorna un array con los elementos empleando el delimitador como separador entre ellos,

"Hola a todos".split(" ");

>(3) ["Hola", "a", "todos"]

[1,2,3,4].join()

>"1,2,3,4"

[1,2,3,4].join("-")

>"1-2-3-4"

// fill(valor[,comienzo[,fin]) :. Rellena los elementos de la matriz entre comienzo (0 si no se especifica) y fin con el valor indicado

[1,2,3,4,5].fill(0)

>(5) [0, 0, 0, 0, 0]

[1,2,3,4,5].fill(2,2,3)

>(5) [1, 2, 2, 4, 5]

[1,2,3,4,5].fill(2,2)

>(5) [1, 2, 2, 2, 2]
```

NOTA : La clase `Array` tiene un método estático `isArray` que retorna true si el objeto que se le pasa es una matriz

```
typeof arr

>"object"

Array.isArray(arr)

>true
```